

SW Engineering CSC 648/848 -Spring 2026 Section 02

Team 01, Milestone 4

4/21/2026

Project Name: LifeTrack

Team Members

Khaterina Sengchareune - Team lead & Github Master

ksengchareune@sfsu.edu

Mohit Kumar - Database Engineer

mkumar3@sfsu.edu

John Narnaja - Backend Engineer

jnaranja@sfsu.edu

Alex Hernandez - Backend Engineer

ahernandez75@sfsu.edu

Will Brust - Frontend Engineer

wbrust@sfsu.edu

Lasiru Weerasuriya - Frontend Engineer

lweerasuriya@sfsu.edu

Revisions Made	Date submitted	Date Revised

1. Product Summary

Name of product:

LifeTrack

Itemize List of all major committed functions:

- Home page
- Search (including search field validation)
- Search Results
- Personal Dashboard creation (to manage goals, projects, learning activities, and development areas)
- Data Upload
- Dashboard (user & admin)
- UI responsiveness
- Performance

Uniqueness

LifeTrack is a unified student wellness and life management platform, not just a resource directory. Most campus tools are either resource finders or personal dashboards, not both in one place. We are combining a searchable wellness resource database, a personal profile with academic info, goal tracking, mood/wellness logging, career, and productivity sections.

URL

<http://34.83.218.226/>

2. Usability Test Plan

Test Objectives:

The object of this test is to evaluate the search functionality of the Team 1 Student's resources. This test specifically assesses how effectively and intuitively new users can locate wellness resources using the search feature, including entering queries, interpreting results, and applying filters.

The search function is the primary mechanism through which students discover resources on the platforms. Users with varying levels of technical comfort depend on it, so the search experience must be clear, fast, and yield relevant results without confusion. This test is being conducted at the milestone 4 state to gather user satisfaction data before the final feature completion, allowing the team to identify and address usability issues while there is still time to iterate. The test focuses on user satisfaction as measured by subject Likert-scale responses following a structured task. Findings will inform the team's final UI and interaction design before Milestone 5.

Test Background and setup:

System Setup:

The application is a [Node.js/Express](#) web application using a MySQL database, hosted on a cloud-based deployment server. Tester will access the live system using a standard web browser on a laptop or desktop computer. No installation is required. The application must be running and accessible before the start of each test session. The facilitator should verify the deployment is live and that search results are populated before the test begins.

Starting Point:

The tester should be logged into an account. On the dashboard, the search button should be on the top navigation bar (top right). No prior context about the site should be given to the tester beyond what is stated in the task description below.

Intended Users:

The intended users for this test are SFSU students who are unfamiliar with the application. Tester should represent the general student population; no prior experience with the platform is required or desired.

Usability Task Description

Task: You are a student looking for academic resources. Using the search feature on the homepage, search for a relevant source (type academic in the search bar and press Search). Review the results that appear. If filters are available, try applying at least one to narrow down the results. Note whether results seem relevant to your query.

After completing the above steps, stop and fill out the questionnaire below

Measure Effectiveness:

Effectiveness will be measured by whether the test successfully locates at least one relevant wellness resource using the function without facilitator assistance. The facilitator will observe and record task completion as a binary outcome (complete/ not completed) for each tester.

Measuring Efficiency:

Efficiency will be measured by recording the time elapsed from when the tester begins typing in the search bar to when they identify a result they consider relevant. The facilitator will use a stopwatch to record this duration in seconds for each tester and calculate the average across all participants.

Likert Subjective Satisfaction Questionnaire:

For each statement below, circle the number that best reflects your level of agreement based on your experience completing the search task above.

Q1: I found it easy to locate a relevant academic resource using the search function.

1-Strongly Agree 2 - Disagree 3- Neutral 4- Agree 5- Strongly Agree

Q2: The search results displayed were relevant and useful to my query

1-Strongly Agree 2 - Disagree 3- Neutral 4- Agree 5- Strongly Agree

Q3: Overall, I am satisfied with the search experience on this application.

1-Strongly Agree 2 - Disagree 3- Neutral 4- Agree 5- Strongly Agree

3. QA Test Plan

Test Objectives:

The objective of this QA test plan is to verify that the search feature for student resources works correctly and consistently on the deployed LifeTrack system. This testing checks whether users can enter valid search terms, receive relevant results, use available filters, and receive proper behavior when no matching results are found.

HW and SW Setup:

Testing was performed on a 16-inch MacBook Pro 2023 with an Apple M2 Pro chip and 16GB of memory, running on macOS Tahoe 26.3.1. The software used for testing included Google Chrome version 147 and Safari on macOS Tahoe 26.3.1. The system under test was the deployed LifeTrack web application, which is a [Node.js](#) and Express application connected to a MySQL database and hosted on the team deployment server.

The application URL used for testing was: <http://34.83.218.226/pages/search.html>

Feature to be Tested:

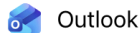
The feature being tested is the resource search functionality on the homepage. This includes entering a search query, submitting the query, viewing search results, and applying available filters to narrow the results.

QA Test Plan Tables

Test #	Test Title	TestDescription	Test Input	Expected Correct Output	Chrome Result	Safari Result
1	Basic	Verify that the	Enter	The system displays	FAIL	FAIL

QA Test Plan Tables						
Test #	Test Title	TestDescription	Test Input	Expected Correct Output	Chrome Result	Safari Result
	keyword search	system returns relevant resources when a valid keyword is entered in the search bar	academic in the search bar and click Search	one or more relevant academic resource results without errors		
2	Search with filter applied	Verify that the user can narrow search results by applying an available filter after searching	Enter academic, click Search, then apply one available filter	The system updates the results and shows only entries that match both the keyword and selected filter	FAIL	FAIL
3	No matching results	Verify that the system handles unmatched queries correctly	Enter a query with no expected matches, such as zzzztestresource, and click Search	The system displays a clear no results message or empty results state without crashing or showing unrelated broken output	PASS	PASS

4. Code Review



Re: milestone 4 code review

From Mohit Kumar <mkumar3@sfsu.edu>
Date Tue 4/21/2026 4:53 PM
To Khaterina Sengchareune <ksengchareune@sfsu.edu>

Good evening, Khaterina,

Thank you again for sending me the code to review. I reviewed the frontend JavaScript code for the search results page for our Milestone 4 code review section. Overall, the code is clear, organized, and directly connected to the search feature we used for our usability and QA testing. I like that the code separates the search loading logic from the result card rendering logic, and I also think the `escapeHtml` helper is a good security practice. I also understand that we are using a simple JavaScript ES6 coding style. The code uses camelCase variable and function names, `async` and `await` for asynchronous API calls, small helper functions for repeated logic, and clear inline comments that explain the purpose of each section.

I added review comments to the code. The main improvements I noticed are that the file header should make sure it starts with a proper opening comment marker and include basic information like the file name, project name, team, and purpose. The code should check that required DOM elements exist before using them, and the frontend could also validate the search query before sending it to the backend. Since our security section says the search bar should allow up to 40 alphanumeric characters, adding frontend validation would help users see errors earlier. I also recommend improving URL validation before rendering external links, because `escapeAttribute` only escapes quotation marks right now. One other small thing to fix would be small typos throughout the comments. These do not break the code, but fixing them will help make the code easier to read and look more professional.

These are not major problems, but they would make the search code safer, cleaner, and easier to maintain. I have included the commented code below, starting with a small header.

These are not major problems, but they would make the search code safer, cleaner, and easier to maintain. I have included the commented code below, starting with a small header.

```
/*
  File: searchResults.js
  Project: LifeTrack
  Team: Team 01
  Purpose:
  Handles the frontend logic for the search results page.
  It reads search parameters from the URL, calls the backend search API,
  and renders loading, error, empty, and result states.
*/

// REVIEW: good use of DOMContentLoaded because it prevents the script from running before
// the page elements exist.
document.addEventListener("DOMContentLoaded", () => {
  const path = window.location.pathname;

  if (path.endsWith("results.html")) {
    loadSearchResults();
  }
});
```

```
}  
});  
  
// REVIEW: clear helper function. This keeps token retrieval separate from the main search  
logic.  
function getAccessToken() {  
  return localStorage.getItem("accessToken");  
}  
  
// REVIEW: this function is the main controller for the search results page.  
// It reads the URL, calls the backend, and updates the page based on the response.  
async function loadSearchResults() {  
  const params = new URLSearchParams(window.location.search);  
  const q = (params.get("q") || "").trim();  
  const category = (params.get("category") || "").trim();  
  
  // REVIEW: possible improvement.  
  // Since Milestone 4 security requires search input to be up to 40 alphanumeric characters,  
  // the frontend could validate q here before sending the request.  
  // Backend validation is still required, but frontend validation would give users faster feedback.  
  
  const resultsSummary = document.getElementById("resultsSummary");  
  const activeFilters = document.getElementById("activeFilters");  
  const loadingState = document.getElementById("loadingState");  
  const errorState = document.getElementById("errorState");  
  const errorMessage = document.getElementById("errorMessage");  
  const emptyState = document.getElementById("emptyState");  
  const resultsGrid = document.getElementById("resultsGrid");  
  
  // REVIEW: possible improvement.  
  // Add a guard clause to make sure these elements exist before using them.  
  // This would prevent errors if the HTML structure changes later.  
  
  const filterParts = [];  
  if (q) filterParts.push(`Keyword: "${q}"`);  
  if (category) filterParts.push(`Category: ${capitalizeWords(category)}`);  
  
  activeFilters.textContent =  
    filterParts.length > 0  
      ? filterParts.join(" • ")  
      : "Showing all available resources";  
  
  try {  
    const apiParams = new URLSearchParams();  
    if (q) apiParams.append("q", q);  
    if (category) apiParams.append("category", category);  
  
    const headers = {};  
    const accessToken = getAccessToken();  
  
    if (accessToken) {  
      headers.Authorization = `Bearer ${accessToken}`;  
    }  
  }  
}
```



```
// REVIEW: good use of fetch with query parameters built through URLSearchParams.
// This is cleaner and safer than manually building the query string.
const response = await fetch(`/api/resources/search?${apiParams.toString()}`, {
  method: "GET",
  headers
});

const data = await response.json();

loadingState.style.display = "none";

if (!response.ok || !data.success) {
  const joinedErrors = Array.isArray(data.errors) ? data.errors.join(" ") : null;
  throw new Error(joinedErrors || data.error || "Search request failed.");
}

const results = Array.isArray(data.results) ? data.results : [];
const total = typeof data.total === "number" ? data.total : results.length;
const isAuthenticated = Boolean(data.isAuthenticated);

if (results.length === 0) {
  resultsSummary.textContent =
    "Search completed, but no matching resources were found.";
  emptyState.style.display = "block";
  return;
}

resultsSummary.textContent = isAuthenticated
  ? `Displaying ${total} result${total === 1 ? "" : "s"} from public and personal resources.`
  : `Displaying ${total} public result${total === 1 ? "" : "s"}.`;

// REVIEW: good separation of rendering logic.
// buildResultCard keeps this part easier to read.
resultsGrid.innerHTML = results.map(buildResultCard).join("");
resultsGrid.style.display = "grid";

} catch (error) {
  loadingState.style.display = "none";
  errorState.style.display = "block";
  errorMessage.textContent =
    error.message || "Something went wrong while loading results.";

  // REVIEW: good error handling for failed backend or network requests.
  // This prevents the user from being stuck on a loading state.
}
}
```

```
// REVIEW: good helper function because each search result card is built in one place.
function buildResultCard(item) {
  const title = escapeHtml(item.title || "Untitled Resource");
  const description = escapeHtml(item.description || "No description available.");
  const category = escapeHtml(item.category || "Uncategorized");
  const source = escapeHtml(item.source || "resource");
  const contentType = escapeHtml(item.content_type || "Unknown Type");
```

```

const imageUrl = item.image_url || "https://placeholder.co/600x340?text=No+Image";
const url = item.url || "#";
const itemId = item.id ?? "N/A";
const safeImageUrl = escapeAttribute(imageUrl);
const safeUrl = escapeAttribute(url);
const canOpenExternal = url && url !== "#";

// REVIEW: possible improvement.
// itemId is created but not displayed anywhere.
// Either display it if needed for debugging, or remove it to keep the code cleaner.

// REVIEW: possible security improvement.
// escapeAttribute helps with quotes, but external URLs should also be validated.
// For example, only allow http or https URLs before rendering them as clickable links.

return `
<article class="card result-card">
  

  <div class="result-content">
    <div class="result-meta">
      <span class="pill">${capitalizeWords(category)}</span>
      <span class="pill">${capitalizeWords(source)}</span>
      <span class="pill">${capitalizeWords(contentType)}</span>
    </div>

    <h2>${title}</h2>
    <p>${description}</p>

    <div style="margin-top:14px;">
      ${
        canOpenExternal
        ? `<a class="btn" href="{safeUrl}" target="_blank" rel="noopener noreferrer">View
Resource</a>`
        : `<button class="btn secondary" type="button" disabled>No External Link</button>`
      }
    </div>
  </div>
</article>
`;
}

// REVIEW: helpful display helper.
// It improves readability for labels such as community_shared.
function capitalizeWords(text) {
  return String(text)
    .split(" ")
    .join(" ");
}

```

```
.replace(/\b\w/g, (char) => char.toUpperCase());
}

// REVIEW: good security helper.
// This reduces the risk of HTML injection when displaying backend text.
function escapeHtml(value) {
  return String(value)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}

// REVIEW: this is useful, but it should be expanded.
// It currently escapes quotes only, so URL validation should also be added.
function escapeAttribute(value) {
  return String(value).replace(/"/g, "&quot;");
}

Best,
Mohit Kumar
918382106
```

From: Khaterina Sengchareune <ksengchareune@sfsu.edu>
Sent: Tuesday, April 21, 2026 3:10 PM
To: Mohit Kumar <mkumar3@sfsu.edu>; Mohit Kumar <mkumar3@sfsu.edu>
Subject: milestone 4 code review

```
/*
  Handles the search results page for the frontend search
  it reads query parameters from results.html, calls the backend search API,
  and renders loading/error/empty/result states.
*/

//waits till page has finished loading before running JS
//Stops errors if JS accesses elements before they exist in DOM
document.addEventListener("DOMContentLoaded", () => {
  const path = window.location.pathname;

  if (path.endsWith("results.html")) {
    loadSearchResults();
  }
});

//reads the saved token so logged in users can access personal search results
function getAccessToken() {
  return localStorage.getItem("accessToken");
}

//Reads search values from URL & sends them to abckend,
//then updates page based on the response
async function loadSearchResults() {
  //grab query string from URL (if they dont exist use empty string)
```

```
const params = new URLSearchParams(window.location.search);
const q = (params.get("q") || "").trim();
const category = (params.get("category") || "").trim();

//reference to HTML elements which we will update dynamically
const resultsSummary = document.getElementById("resultsSummary");
const activeFilters = document.getElementById("activeFilters");
const loadingState = document.getElementById("loadingState");
const errorState = document.getElementById("errorState");
const errorMessage = document.getElementById("errorMessage");
const emptyState = document.getElementById("emptyState");
const resultsGrid = document.getElementById("resultsGrid");

//build text summary of current filters used
const filterParts = [];
if (q) filterParts.push(`Keyword: "${q}"`);
if (category) filterParts.push(`Category: ${capitalizeWords(category)}`);

//if no filters, show all public resources
activeFilters.textContent =
  filterParts.length > 0
    ? filterParts.join(" • ")
    : "Showing all available resources";

//build api query param that is sent to backend
//include q & category if user provided them
try {
  const apiParams = new URLSearchParams();
  if (q) apiParams.append("q", q);
  if (category) apiParams.append("category", category);

  const headers = {};
  const accessToken = getAccessToken();

  //If a token exists, include it so the backend can return personal results
  if (accessToken) {
    headers.Authorization = `Bearer ${accessToken}`;
  }

  //GET request to backend search route
  //convert backend JSON text to JS object
  const response = await fetch(`/api/resources/search?${apiParams.toString()}`, {
    method: "GET",
    headers
  });

  const data = await response.json();

  //request finished --> hide loading box
  loadingState.style.display = "none";

  //if http request failure or backend success = false
  //treat as error & jump to catch block
  if (!response.ok || !data.success) {
```

```

const joinedErrors = Array.isArray(data.errors) ? data.errors.join(" ") : null;
throw new Error(joinedErrors || data.error || "Search request failed.");
}

//results is always treated as array, if missing use empty array
//Use total from backend (if exists), otherwise count results array
const results = Array.isArray(data.results) ? data.results : [];
const total = typeof data.total === "number" ? data.total : results.length;
const isAuthenticated = Boolean(data.isAuthenticated);

//no matching results found --> show message
if (results.length === 0) {
  resultsSummary.textContent =
    "Search completed, but no matching resources were found.";
  emptyState.style.display = "block";
  return;
}

//Update the top summary depending on whether the user was authenticated
resultsSummary.textContent = isAuthenticated
  ? `Displaying ${total} result${total === 1 ? "" : "s"} from public and personal resources.`
  : `Displaying ${total} public result${total === 1 ? "" : "s"}.`;

//turn each result into html card w/ buildResultCard, combine into 1 string & place in grid
//make sure results grid is visible since cards are rdy
resultsGrid.innerHTML = results.map(buildResultCard).join("");
resultsGrid.style.display = "grid";

//if something goes wrong/error (network/backend/etc.) hide loading & show error box
} catch (error) {
  loadingState.style.display = "none";
  errorState.style.display = "block";
  errorMessage.textContent =
    error.message || "Something went wrong while loading results.";
}
}

//build one result card using html string, each item from backend turns into one visual card on
page
function buildResultCard(item) {

  //prepare text values (safely/securely) so unexpected chars dont break html
  const title = escapeHtml(item.title || "Untitled Resource");
  const description = escapeHtml(item.description || "No description available.");
  const category = escapeHtml(item.category || "Uncategorized");
  const source = escapeHtml(item.source || "resource");
  const contentType = escapeHtml(item.content_type || "Unknown Type");
  //if image provided exists use it, if not use placeholder img
  const imageUrl = item.image_url || https://placeholder.co/600x340?text=No+Image;
  //use resource link if exists, otherwise use # so btn does not break
  const url = item.url || "#";
  const itemId = item.id ?? "N/A";
  const safeImageUrl = escapeAttribute(imageUrl);
  const safeUrl = escapeAttribute(url);

```

```

const canOpenExternal = url && url !== "#";

//return full html for one result card
return `
<article class="card result-card">
  

  <div class="result-content">
    <div class="result-meta">
      <span class="pill">${capitalizeWords(category)}</span>
      <span class="pill">${capitalizeWords(source)}</span>
      <span class="pill">${capitalizeWords(contentType)}</span>
    </div>

    <h2>${title}</h2>
    <p>${description}</p>

    <div style="margin-top:14px;">
      ${
        canOpenExternal
        ? `<a class="btn" href="${safeUrl}" target="_blank" rel="noopener noreferrer">View
Resource</a>`
        : `<button class="btn secondary" type="button" disabled>No External Link</button>`
      }
    </div>
  </div>
</article>
`;
}

//helper to amke values display better
//i.e. "community_shared" = "Community Shared"
function capitalizeWords(text) {
  return String(text)
    .split("_")
    .join(" ")
    .replace(/\b\w/g, (char) => char.toUpperCase());
}

//prevents html injextion by replacing special chars w/ safe html versions
function escapeHtml(value) {
  return String(value)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}

```

```
//escape quotes for vals that are placed inside html attributes like src="" or href=""  
function escapeAttribute(value) {  
  return String(value).replace(/"/g, "&quot;");  
}
```

5. Self-check on best practices for security

List major assets you are protecting:

Database:

All incoming HTTP requests pass through authentication middleware before any database interaction occurs. All SQL queries use parameterized statements to prevent SQL injection attacks.

User identity:

User sessions are managed with JSON Web Tokens; a short-lived access token and a refresh token are issued at login. Protected API routes require a valid bearer token in the authorization header before any user data is returned or modified.

Passwords:

All user passwords are hashed using bcrypt before being stored in the database. Plaintext passwords are never saved or logged at any point.

File Uploads:

Profile image uploads are validated server-side to permit only JPEG. MIME type and file size are enforced by the upload middleware before the file is accepted and stored.

Encrypted passwords:

Team 01 confirms we are using encrypted passwords in the database.

Input Data Validation:

Team 01 confirms we have input data validation implemented in our codebase.

- Search bar input: validated on the backend to accept up to 40 alphanumeric characters only, using a regex check before the query is executed
- Password fields: enforced minimum length and complexity requirements at registration.
- Profile text inputs: trimmed and sanitized before being passed to parameterized queries.
- File uploads: MIME type validated server side(JPG only), file sized capped at 5MB, frontend HTML output is escaped using a custom function to prevent XSS.

6. Self-check: Adherence to original Non-functional specs – performed by team leads

1. Application shall be developed, tested, and deployed using tools and servers approved by ClassCTO and as agreed in M0. **- DONE**
2. Application shall be optimized for standard desktop/laptop browsers and must render correctly on the two latest versions of at least two major browsers. **- DONE**
3. Selected application functions must render well on mobile devices (this is a plus). **-DONE**
4. Data shall be stored in the team's chosen database technology on the team's deployment server. **- DONE**
5. Privacy of users shall be protected, and all privacy policies shall be clearly communicated to Users. **- IN PROGRESS**
6. The language used shall be English. **- DONE**
7. Application shall be very easy to use and intuitive. **- DONE**
8. Google Maps and Google Analytics shall be added where appropriate (e.g., campus resources, study locations, event discovery, usage analytics). **-- NOT APPLICABLE (WE ARE NOT IMPLEMENTING AI INTO OUR APP)**
9. No external e-mail clients shall be allowed. You shall use InSite webmail only. **- DONE**
10. Pay functionality, if any, shall not be implemented nor simulated in the UI. **- DONE**
11. Site security: basic best practices shall be applied (as covered in the class). **- DONE**
12. Modern Software Engineering processes and practices shall be used as specified in the class, including collaborative and continuous software development. **-DONE**
13. The website shall prominently display the following exact text on all pages at the top of the

WWW page:"SFSU Software Engineering Project CSC 648-848, Spring 2026. For DemonstrationOnly" **- IN PROGRESS**

14. AI-powered assistant and intelligent search enhancements are optional (bonus requirements)and must be implemented modularly. **- NOT APPLICABLE (WE ARE NOT IMPLEMENTING AI INTO OUR APP)**